SNOW BROS

Object-Oriented Design Report
& UML Class Diagram Documentation

Muhammad Abdullah Saqib
Zubair Ashraf

CS Object-Oriented Programming Final Project 2025

1. Project Overview

Snow Bros is a 2D arcade-style game built in C++ using SFML (Simple and Fast Multimedia Library). The game faithfully recreates classical mechanics, players throw snowballs to freeze enemies, then kick them to send rolling snowballs that chain-kill other enemies. The project spans ten levels with multiple enemy types, two-player co-op, a persistent leaderboard, and a complete authentication system.

2. The Four OOP Pillars Applied to Snow Bros

2.1 Encapsulation

Encapsulation means bundling data and the methods that operate on it inside a single class, and hiding internal details behind a public interface.

Enemy class protected state, controlled access: All enemy attributes (position, speed, health, alive flag, snow-cover state, direction) are declared protected in the Enemy base class. External code cannot touch them directly; it must go through public methods like damage(float), getZinda(), getCover(), and kick(float). This means Collision.h can apply damage and query state without knowing anything about how health is stored internally.

Player internal state hidden behind interface: The Player class hides its lives, score, gems, hit, hitTimer, Ground, jumpSpeed, and the 10-element SnowBall ball[10] array as private members. External code only calls lifedown(), addScore(int), getBallActive(int), setBallActive(int, bool), etc.

AuthSystem credential logic sealed: The hashPassword(std::string) method is private. The file I/O for reading and writing users.txt is fully contained inside the class. Callers only see login() and signup(), which return simple booleans.

Leaderboard file persistence encapsulated: The loadFromFile() and sort logic are private. External code only calls updateScore(string, int) and draw(window).

2.2 Inheritance

Inheritance allows derived classes to reuse and extend the behaviour of their parent. Snow Bros uses a four-level deep enemy hierarchy:

Enemy (abstract base): Defines the contract for all enemies. Contains shared attributes (position, speed, health, zinda flag, snow-cover state, sprite, texture). Declares virtual void update(Platform[], int) = 0 and virtual void draw(RenderWindow&) = 0 as pure virtual functions, making Enemy an abstract class, so that no direct instantiation is possible.

Bottom: Enemy, the ground-walking enemy. Adds jumpE, groundE, and animation frame tracking. Implements update() with gravity, platform collision, directional walking, and snow-cover state machine.

FlyingFooga: Botom inherits Botom's ground behaviour, adds flyingE, flyx/flyy velocity, and flyTime. Periodically launches into a flying phase.

Tornado FlyingFooga inherits flight behaviour, adds a knife projectile: sf::RectangleShape knife, knifeTime, zindaK. Throws a knife aimed at the player's position.

InvisibleEnemy: Tornado extends Tornado. Overrides update() to toggle visibility based on flying state, and overrides draw() to either render or hide the sprite.

Mchild: Bottom is a simpler extension of Bottom with a different jump/fall pattern, used in boss encounters.

Mogera and Gama: Enemy boss classes that inherit directly from Enemy and implement entirely custom update() and draw(). Gama manages five directional rocket projectiles (top, bottom, left, right, centre) and a health bar.

Player: sf::Drawable inherits from SFML's sf::Drawable abstract class, overriding draw(sf::RenderTarget&, sf::RenderStates) so the player can be passed directly to window.draw(player).

2.3 Polymorphism

Polymorphism lets the main loop treat all enemies through a single Enemy* pointer, regardless of their actual type.

Runtime: In main.cpp, a unified Enemy* enemies[20] array is populated with pointers to Botoms, FlyingFoogas, Tornados, InvisibleEnemies, Mchilds, and bosses. When level.update(player) internally calls enemy->update() or enemy->draw(), C++'s vtable dispatch automatically calls the correct overridden method for each concrete type.

draw() polymorphism: InvisibleEnemy::draw() conditionally calls Tornado::draw() or skips rendering entirely when the enemy is in its invisible flying state. The Collision system queries enemy->getZinda() and enemy->getCover() polymorphically; the same code path handles every enemy type.

update() overloading: Enemy provides a default two-parameter update(Platform[], int) and a three-parameter update(Platform[], int, sf::Vector2f) version for enemies that track the player position (Tornado, InvisibleEnemy). The three-parameter version has a default implementation that calls the two-parameter one, so derived classes only override what they need.

sf::Drawable polymorphism: Because Player extends sf::Drawable and overrides draw(), both player and player2 can be passed to window.draw() directly SFML calls the correct virtual draw internally.

2.4 Abstraction

Abstraction means exposing only what callers need and hiding complexity behind a well-named interface.

Enemy as abstract class: The pure virtual update() and draw() in Enemy create a contract every enemy type must fulfil. No caller ever needs to know the internal walking, flying, or attack logic; they only call update() and draw().

Platform abstraction: Platform wraps an sf::RectangleShape and exposes only getBounds() and draw(window). The debug hitbox rendering (blue outline when H is held) is entirely internal callers are not aware of it.

Snowball abstraction inside Player: The SnowBall ball[10] array management (activation, deactivation, bounds checking) is internal to Player. Collision.h accesses it only through getBallBounds(int), getBallActive(int), and setBallActive(int, bool).

Screen classes: LoginScreen, SignupScreen, MainMenu, LevelSelect, SplashScreen each expose only update(event) returning an integer state-transition code, and draw(window). main.cpp is entirely decoupled from their internal layouts.

3. Class Hierarchy & Relationships

3.1 Enemy Inheritance Tree

The hierarchy has five levels and represents a textbook example of incremental extension:

Class	Inherits From	Key Additions
Enemy	(abstract base)	position, speed, health, zinda, cover, state, snowball reaction, virtual update/draw
Botom	Enemy	jumpE, groundE, animation frames, platform gravity, directional walk
FlyingFooga	Botom	flyingE, flyx, flyy, flyTime, periodic launch into the air
Tornado	FlyingFooga	knife shape, knifeTime, zindaK, aimed knife throw
InvisibleEnemy	Tornado	See flag, toggle visibility on fly state
Mchild	Botom	jumpMC, groundMC, simpler jump pattern
Mogera	Enemy	boss with health bar and Mchild spawning
Gama	Enemy	5 directional rockets, a rocket health bar, and attack phases

3.2 Other Class Relationships

Composition Player HAS-A SnowBall[10]: Player owns its snowballs directly. When a Player object is destroyed, all snowballs go with it.

Composition Gama HAS-A rocket[5]: Gama owns and manages all five rockets internally.

Association Player uses Enemy* (via Collision): Player's update(Enemy*[], int) accepts an array of enemy pointers. This is a non-owning reference Player does not manage the lifetime of enemies.

Association LevelSystem HAS enemy instances: LevelSystem stores concrete Bottom, FlyingFooga, Tornado, InvisibleEnemy arrays by value, providing raw pointers to the main loop.

Association AuthSystem and Leaderboard share file persistence: Both read and write separate text files (users.txt, leaderboard.txt). They are independent systems that cooperate through the game state machine in main.cpp.

3.3 Design Patterns

State Pattern (Game Loop): The gameState integer in main.cpp acts as a classic state machine: 0=Splash, 1=Login, 2=Signup, 3=MainMenu, 4=LevelSelect, 5=Playing, 6=Leaderboard. Each state has its own update and draw logic, and transitions are triggered by return codes from screen classes.

State Pattern (Enemy Snow Cover): The state member inside Enemy implements a 5-state machine: 0=Normal, 1=Half Encased, 2=Fully Encased, 3=Rolling, 4=Melting. Transitions are driven by hitByBall() and kick(float).

Strategy-like Pattern (Collision): The Collision class provides three static methods, BottomCollision, FlyingFoogaCollision, and TornadoCollision, that implement different collision strategies for each enemy category. This keeps collision logic separate from both Player and Enemy.

Observer-like Scoring: The main loop stores the enemy alive-state before each update, then compares after if an enemy transitioned from alive to dead, a score bonus is awarded. This is a lightweight observer pattern, avoiding tight coupling between Enemy and Player score.

4. Code Modularity and Reusability

The project achieves high modularity through a header-per-class structure and carefully scoped responsibilities.

Single Responsibility: Each class does one thing. The platform handles geometry and debug rendering. AuthSystem handles credential validation. HUD handles score/life display. None of these classes knows about the others.

Reusable Enemy Base: All collision logic in Collision.h operates on getBounds(), getZinda(), getCover(), and damage(), all defined in Enemy. Adding a new enemy type requires zero changes to Collision.h.

Reusable Screen Interface: All screen classes expose the same update(sf::Event&) / draw(sf::RenderWindow&) contract. The game loop treats them uniformly.

setVariantTexture: The Enemy::setVariantTexture(std::string) method lets any derived class swap its sprite at runtime, useful for enemy variants without creating new subclasses.

Multiplayer reuse: The Player class is fully reused for Player 2. The setPlayer2(bool) method switches the texture to Naruto. Both players share identical physics, snowball, and collision logic, with zero code duplication.

5. Non-OOP Aspects and Justifications

Not every aspect of the project is purely OOP. We acknowledge these pragmatic decisions:

Raw arrays for enemies: The main loop uses a raw Enemy* enemies[20] array and C-style index loops.

Global game state integer: The gameState variable in main.cpp is a procedural state machine.

Static methods in Collision: Collision is a utility class with all-static methods, effectively a namespace. This avoids instantiation overhead and is a pragmatic choice for a stateless utility.

File I/O in AuthSystem and Leaderboard: Plaintext files (users.txt, leaderboard.txt) are used for persistence. A database or serialisation library would be safer, but is out of scope for this project.

Personal Learnings

- The value of pure virtual functions: being forced to implement update() and draw() in every enemy class caught missing cases early.
- Copy constructors and assignment operators in C++ are non-trivial. The Enemy copy constructor manually rebinds the sprite texture to the copied texture to avoid dangling pointers.
- Managing lifetime with raw pointers requires discipline, the enemies[] pointer array in main.cpp must never outlive the level's enemy storage
- SFML's event system only delivers one event per frame in a single-event model; we had to store the last event per frame carefully.
- Working with a game loop taught us about frame-rate independence, delta time, and why clamping at 60 FPS stabilises physics

7. Project Reflection & Potential Improvements

7.1 What Went Well

- The enemy hierarchy scaled beautifully, and adding InvisibleEnemy required almost no changes to existing code.
- The screen-based state machine kept the main.cpp readable and screens independently testable.
- The snow-cover state machine is clean, deterministic, and fun to interact with. Half-encased enemies melting back is a satisfying mechanic.

- Multiplayer was achievable with almost no additional logic because Player was designed generically from the start

8. Challenges Faced

8.1 SFML sprite/texture ownership: sf::Sprite does not own its sf::Texture. When objects were copied (e.g., level reloads), sprites pointed to freed textures fixed by implementing custom copy constructors that rebind the sprite to the new texture.

8.2 The Collision class grew per enemy type. A more scalable approach would be a visitor pattern or a collision component attached to each enemy.

8.3 The Gama boss's rocket direction-setting is done via a switch on the index, which is fragile. A data-driven approach (table of offsets) would be cleaner.

8.4 Score is awarded to both players simultaneously on a kill in true co-op; attribution should be tracked per player

9. Potential Improvements

9.1 Use of enum

9.2 Introduce a Component/ECS architecture for enemies to compose behaviours (walking, flying, shooting) instead of inheriting them.

9.3 Use a SceneManager class to formalise the state machine and make screen transitions data-driven.

9.4 Replace the integer hash with bcrypt or SHA-256 for password security.

9.5 Add an event/observer system for scoring so Player and Enemy are fully decoupled.

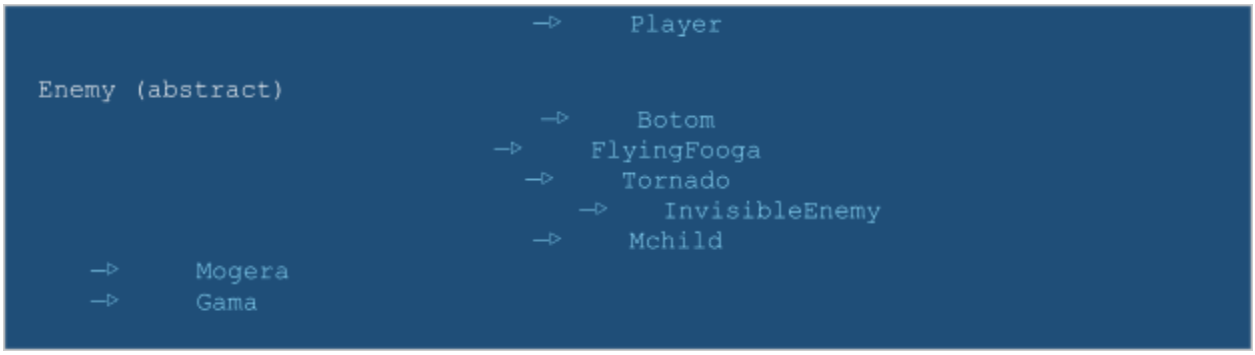
9.6 Implement save/load of progress using binary serialisation rather than reconstructing from the level number alone.

9.7 Increase Temwork

10. UML Class Diagram Textual Representation

The following tables present the UML class relationships. A visual diagram is described structurally below.

10.1Inheritance Relationships (—▷ = inherits)



10.2Composition Relationships (◆ = owns)

Owner	Owned	Multiplicity / Notes
Player	SnowBall	1 Player ◆ SnowBalls
Gama	sf::RectangleShape (rocket)	1 Gama ◆ 5 rockets + 5 bool active flags
Gama	sf::RectangleShape (health bar)	1 Gama ◆ 1 health bar + 1 health background
LevelSystem	Botom[]	1 Level ◆ up to 5 Bottoms

LevelSystem	FlyingFooga[]	1 Level ◆ up to 2 FlyingFoogas
LevelSystem	Tornado[]	1 Level ◆ up to 2 Tornados
LevelSystem	InvisibleEnemy[]	1 Level ◆ up to 5 InvisibleEnemies
LevelSystem	Mogera	1 Level ◆ 1 Mogera (level 5 only)
LevelSystem	Gama	1 Level ◆ 1 Gama (level 10 only)

10.3Association / Usage Relationships (——▸ = uses)

From	To	Notes
main.cpp	All screen classes	Calls update() and draw() on active screen
main.cpp	AuthSystem	Passes to LoginScreen / SignupScreen

main.cpp	Leaderboard	Calls updateScore() after each kill
Player	Enemy*[]	Non-owning; reads bounds for collision
Collision	Player	Reads/writes player bounds, lifedown()
Collision	Enemy subtypes	Calls damage(), getZinda(), getCover()
HUD	Player (values)	Reads score, lives, gems via getters

Question: You currently have 10 levels and 5 different enemy types. How would you scale the game to 30 levels and 4

additional game modes? Describe your OOP-based approach in the report.

Only have to add code in the level system.cpp as my platforms are manually coded, the rest of the proper OOP structure is followed, so no big change is required.

Zero changes to LevelSystem, Collision, or main.cpp.

Our draw.io diagram already shows a **GameState** abstract class with update(), draw(), handleEvent(), onEnter(), and onExit(). Each game mode becomes a subclass of GameState. The StateManager holds the current state and delegates all calls to it. Switching game modes is a single changeState(new Mode()) call.

Every existing class (Player, Enemy and all subclasses, Leaderboard, Platform) remains untouched. Each new mode is an isolated GameState subclass that composes existing systems in a new way.